

Statose Football Data Platform — Schema Design Document

Overview

This document explains the database schema for Statose, a football analytics platform, and the reasoning behind each design decision. The schema is built to support:

- Cross-league player scouting
 - Player progression tracking over seasons
 - Multi-provider data ingestion (Gradient, ScoutLab, FotMob)
 - Percentile-based analytics and comparisons
 - A metric system that scales without schema changes
 - Planned future support for **team metrics**, **game/match metrics**, and **transfers** (see Section 8)
-

1. Core Design Philosophy

1.1 Metric-Based Architecture

Most sports databases hardcode stats as columns: `goals`, `assists`, `xg`, `tackles_won`, etc. The problem with that approach is every time a data provider adds a new stat, you need a migration to add a column. With dozens of providers and hundreds of possible metrics (and more being invented constantly in football analytics), that doesn't scale.

Instead, Statose uses a **metric dictionary** + **value store** pattern, similar to how a key-value store works, but with structure:

- `metrics` = the *definition* of a stat (what it is, who provides it, how to interpret it)
- `player_metrics` = the *actual values* (what a specific player did, in a specific season)

Why this matters in practice

If FotMob starts providing a new stat tomorrow — say "progressive carries into the box" — you don't touch the schema at all. You just insert a new row into `metrics`, and your ingestion script starts writing values into `player_metrics` against that new `metric_id`. No migration, no downtime, no code changes to the table structure.

The tradeoff: querying is slightly less ergonomic than `SELECT goals FROM player_stats` — you instead join through `metrics` and filter by `metric_key`. This is a reasonable tradeoff for a system designed to ingest from multiple unpredictable providers.

1.2 Separation of Identity, Context, and Performance

The schema is split into three conceptual layers. Keeping these separate is what makes the system flexible — each layer can evolve independently.

Identity Layer — "who/what exists"

- `countries`
- `players`
- `teams`
- `competitions`

These describe entities that exist independent of time. A player is a player whether or not they're currently playing. A team exists whether or not the season has started.

Context Layer — "where and when"

- `seasons`
- `player_seasons`
- `player_season_competitions`

These connect identities to a specific point in time — which team a player was on, during which season, in which competitions.

Performance Layer — "what happened"

- `metrics`
- `player_metrics`

These store the actual statistical output, attached to a specific context (a `player_season`).

This separation means, for example, that a player's birth date or nationality doesn't need to be re-entered every season, and a metric definition doesn't need to be duplicated per player — it's defined once and referenced many times.

2. Identity Layer — Detailed

2.1 Countries

Represents nations — used both for player nationality and for where a team/competition is based.

Field	Purpose
<code>name</code>	Full country name, must be unique
<code>iso2</code> / <code>iso3</code>	Standard ISO country codes — useful for flag icons, matching against external APIs that use ISO codes instead of names

Why a separate table instead of a string column? Countries are referenced from three places (`players`, `teams`, `competitions`). Storing it once and referencing by ID avoids inconsistent spellings ("USA" vs "United States" vs "US") scattered across the database.

2.2 Teams

Represents football clubs globally — not tied to a specific season.

Field	Purpose
<code>fotmob_id</code>	External ID for matching against FotMob's API during ingestion
<code>name</code> / <code>short_name</code>	Display name and abbreviated form (e.g. "Manchester United" / "Man Utd")
<code>country_id</code>	Which country the club is based in
<code>foundation_year</code> , <code>stadium_name</code> , <code>logo_url</code>	Display/metadata fields

Design decision: Teams are global, not season-specific. A team like Arsenal doesn't get a new row every season — it's one row, forever. Anything that changes season-to-season (which players are on the roster, how the team performed) is tracked through `player_seasons` and (eventually) `team_seasons`, not by duplicating the team itself.

2.3 Players

Represents football players as individuals, independent of any team or season.

Field	Purpose
<code>full_name</code> , <code>date_of_birth</code> , <code>nationality_id</code>	Core identity fields
<code>primary_position</code> , <code>secondary_position</code>	The player's general/career position (e.g. a winger who sometimes plays fullback)
<code>preferred_foot</code> , <code>height_cm</code> , <code>weight_kg</code>	Physical attributes

Design decision: physical attributes in SI units only. No `height_in` or `weight_lbs` columns — if the frontend needs imperial units for display, convert at the application layer. Storing both creates a sync problem (what happens if one gets updated and not the other?) for zero benefit.

Note on position fields: `players.primary_position` is the player's general/default position across their career. This is intentionally separate from `player_seasons.position`, which captures the position they actually played *that specific season* — useful when a manager redeploys a player (e.g. a winger moved to wingback for a season). Both fields exist on purpose, not as duplication.

2.4 Competitions

Represents leagues, cups, and tournaments.

Field	Purpose
<code>country_id</code>	Which country the competition is primarily associated with
<code>gradient_league_id</code> , <code>fotmob_league_id</code>	External IDs for matching during ingestion from each provider
<code>competition_type</code>	'league', 'cup', 'international', etc.
<code>gender</code>	Men's/women's competitions are tracked separately

Design decision: provider IDs are nullable. Not every competition will be indexed by every provider on day one — Gradient might cover a league before FotMob does, or vice versa. Making these `NOT NULL` would block ingesting a competition until *all* providers had data for it, which is backwards. They're nullable and get filled in as each provider's coverage is confirmed.

A note on scaling this further: right now, provider IDs live as columns directly on `teams` and `competitions`. This works fine for 2-3 providers. If a third or fourth provider is added later and this pattern keeps repeating, it may be worth consolidating into a single generic mapping table:

```
sql

CREATE TABLE external_ids (
  id          SERIAL PRIMARY KEY,
  entity_type VARCHAR(30) NOT NULL, -- 'team', 'competition', 'player'
  entity_id   INTEGER NOT NULL,
  provider    VARCHAR(50) NOT NULL,
  external_id VARCHAR(100) NOT NULL,
  UNIQUE (entity_type, entity_id, provider)
);
```

This isn't implemented in v1 — it adds a layer of indirection that isn't justified until the pain of adding more provider-ID columns is actually felt. Mentioned here so the migration path is documented if/when it's needed.

3. Context Layer — Detailed

3.1 Seasons

Represents a football time period (e.g. "2024/25").

Field	Purpose
<code>name</code>	Unique label, e.g. <code>'2024/25'</code>
<code>start_date</code> , <code>end_date</code>	Actual calendar boundaries
<code>is_current</code>	Flag for "the season currently being ingested/displayed"

Design decision: seasons are global, not per-competition. A single `2024/25` row is shared by every competition. This keeps season-based queries ("show me everything from last season") simple, rather than needing to know which competition-specific season label applies.

3.2 PlayerSeasons

The most important table in the context layer — it's the anchor that connects a player to a team for a given season.

Field	Purpose
<code>player_id</code> , <code>team_id</code> , <code>season_id</code>	The core link
<code>shirt_number</code> , <code>position</code>	As-played details for this season
<code>appearances</code> , <code>starts</code> , <code>minutes</code> , <code>goals</code> , <code>assists</code> , <code>yellow_cards</code> , <code>red_cards</code>	Season-aggregate totals

Design decision: supports mid-season transfers. A player can have multiple `player_seasons` rows in the same `season_id` if they transferred — one row per team they played for. This is why there's no unique constraint on `(player_id, season_id)` alone.

Constraint added: `UNIQUE (player_id, team_id, season_id)` — this prevents the one duplicate scenario that *shouldn't* happen: the same player, same team, same season being inserted twice (e.g. if an ingestion script accidentally runs twice). It still allows multiple rows for the same player across *different* teams in the same season, which is the transfer case.

3.3 PlayerSeasonCompetitions

Breaks a `player_season` down by individual competition.

Why this is separate from `player_seasons`: A player's season totals often blend league and cup performance. If you want to know "how did this player do *in the league specifically*, separate from cup games," you need this breakdown. `player_seasons` holds the season-wide totals; this table holds the per-competition slice.

Constraint added: `UNIQUE (player_season_id, competition_id)` — one row per competition per player-season, supporting safe re-ingestion (upsert instead of duplicate insert).

4. Performance Layer — Detailed

4.1 Metrics (Definition Table)

The dictionary of every possible stat the system knows about.

Field	Purpose
<code>provider</code>	Which data source defines this metric (<code>'gradient'</code> , <code>'fotmob'</code> , etc.)
<code>metric_key</code>	The raw identifier as it comes from that provider's API
<code>metric_name</code>	Human-readable name for display
<code>group_name</code>	Category for UI grouping — "Passing", "Shooting", "Defending"
<code>stat_type</code>	<code>'count'</code> , <code>'percentage'</code> , or <code>'rate'</code>
<code>is_per90</code>	Whether this metric is naturally a per-90-minutes rate
<code>is_percentage</code>	Whether to format/display as a %
<code>is_negative</code>	Whether a <i>lower</i> value is actually better (e.g. miscontrols, errors leading to goals) — important for percentile coloring in the UI, so you don't accidentally show a high "errors" percentile in green

Constraint: `UNIQUE (provider, metric_key)` — the same provider can't define the same raw metric key twice. Different providers *can* have overlapping metric concepts (e.g. both Gradient and FotMob might have something like "progressive passes") — they get separate rows because their calculation methods may differ slightly, and you don't want to silently merge numbers that aren't actually computed the same way.

4.2 PlayerMetrics (Fact Table)

The actual values.

Field	Purpose
<code>player_season_id</code> , <code>metric_id</code>	What player-season this value belongs to, and which metric it is
<code>value</code>	The raw stat total for the season
<code>per90</code>	Normalized per-90-minutes version
<code>percentile</code> , <code>percentile_per90</code>	Where this player ranks (0-100) against a comparison population — the core of the scouting feature

Constraint: `UNIQUE (player_season_id, metric_id)` — a player can only have one value per metric per season. This is what makes ingestion idempotent: re-running the pipeline does an `UPSERT` against this constraint instead of creating duplicate rows.

Precision note: `value` and `per90` use `DECIMAL(10,3)` rather than an unbounded `DECIMAL` or a `FLOAT`. Football stats rarely need more than 2-3 decimal places of precision, and using `FLOAT` risks small rounding errors that look wrong when displayed (e.g. `0.300000000000004`). `percentile` uses `DECIMAL(5,2)` since it's always between 0.00 and 100.00.

5. Ingestion System Design

5.1 Data Flow

1. Fetch provider data (Gradient API, etc.)
2. Resolve player identity (match against existing `players` row, or create one)
3. Resolve or create the relevant `player_season` row
4. Flatten the provider's metric groups into individual key-value pairs
5. Map each `metric_key` → a row in `metrics` (auto-creating it if it's new — see 5.2)
6. Upsert into `player_metrics`
7. Optionally store the raw JSON snapshot from the provider (useful for debugging or re-processing later without re-fetching)

5.2 Key Ingestion Principles

Idempotency

Re-running the ingestion pipeline should never create duplicate data. This is enforced at the database level via the unique constraints on `player_seasons`, `player_season_competitions`, and `player_metrics` — ingestion code should always `INSERT ... ON CONFLICT DO UPDATE` against these constraints rather than plain `INSERT`.

Auto-Discovery

When the pipeline encounters a `metric_key` it hasn't seen before from a given provider, it creates a new row in `metrics` automatically. No manual schema change is needed to support a new stat.

Separation of Concerns

The ingestion pipeline's only job is getting data into the database correctly. Percentile calculation, ranking, and any scouting-specific business logic live outside the ingestion layer (e.g. as a separate batch job or computed at query time), not inside the scraper/loader code.

6. Key Database Constraints (Reference)

Table	Constraint
<code>countries</code>	<code>(name, iso2, iso3)</code> all unique
<code>teams</code>	<code>(fotmob_id)</code> unique
<code>seasons</code>	<code>(name)</code> unique
<code>player_seasons</code>	<code>(player_id, team_id, season_id)</code> unique
<code>player_season_competitions</code>	<code>(player_season_id, competition_id)</code> unique
<code>metrics</code>	<code>(provider, metric_key)</code> unique
<code>player_metrics</code>	<code>(player_season_id, metric_id)</code> unique

Indexes added beyond primary/unique keys: Postgres does not automatically index foreign key columns (unlike MySQL). Indexes were added explicitly on every FK column that isn't already covered by a unique constraint, since these are the columns that get filtered/joined on constantly (e.g. "all seasons for this player," "all teams in this country"):

- `teams.country_id`
- `players.nationality_id`
- `competitions.country_id`
- `player_seasons.player_id`, `.team_id`, `.season_id`
- `player_season_competitions.player_season_id`, `.competition_id`
- `player_metrics.metric_id`

7. Why This Design Works

7.1 Scalability

New metrics, new providers, and new competitions can all be added without touching the schema or running a migration.

7.2 Multi-Provider Support

Gradient, ScoutLab, and FotMob data can coexist in the same tables, distinguished by `provider` on `metrics` and external ID columns on `teams`/`competitions`.

7.3 Scouting Power

Percentile fields on `player_metrics` enable cross-league, cross-position comparisons directly from stored data, without recomputing on every page load.

7.4 Historical Tracking

Because `player_seasons` is season-scoped rather than overwritten, a player's full career arc — team by team, season by season — is preserved and queryable.

8. Planned Extensions

These are **not implemented in the v1 schema**, but the existing design was built with them in mind. Here's how each will attach to what already exists, so future work doesn't require reworking the core tables.

8.1 Team Metrics

Mirrors the player metrics pattern, but at the team level.

```
team_seasons      -- team_id, season_id, aggregate team-level stats
                   (wins, draws, losses, goals_for, goals_against, etc.)

team_metrics      -- team_season_id, metric_id, value, per90, percentile
                   (reuses the existing `metrics` table — a metric
                    like "possession %" can apply to both players
                    and teams, just attached to a different fact table)
```

This follows the exact same identity → context → performance pattern already used for players: `teams` (identity, already exists) → `team_seasons` (context, new) → `team_metrics` (performance, new).

8.2 Match / Game-Level Metrics

Adds a granularity level *below* season: the individual match.

```
matches           -- home_team_id, away_team_id, competition_id,
                   season_id, match_date, score

match_player_stats -- match_id, player_id, minutes_played, and a link
                   to per-match metric values (same metrics table,
                   new fact table keyed on match_id + player_id
                   instead of player_season_id)
```

`player_seasons` would then essentially become an aggregate *rollup* of `match_player_stats` for that player-team-season combination, rather than data entered independently. This is a bigger structural addition than team metrics, since it introduces a new time granularity, but it slots in underneath the existing season layer rather than replacing it.

8.3 Transfers

Tracks player movement between clubs over time, independent of season-stat tracking.

```
transfers    -- player_id, from_team_id, to_team_id, transfer_date,  
              fee, transfer_type (permanent/loan/free), window (summer/  
winter)
```

This is largely independent of the rest of the schema — it references `players` and `teams` but doesn't need to touch `player_seasons` at all. It would power a "transfer history" view on a player's profile, separate from their on-pitch stats.

9. Final Summary

This schema forms a **modular football analytics engine** designed to:

- Ingest multi-provider data without manual schema changes
- Normalize inconsistent football statistics into one consistent value store
- Support scalable, percentile-based scouting features
- Track full player career progression across time and leagues
- Extend cleanly into team-level metrics, match-level granularity, and transfer history without reworking the core identity/context/performance structure